

Chương 4

Kiến trúc tập lệnh MIPS

Nội dung

- Kiến trúc tập lệnh
- Khái quát vi xử lý MIPS
- R-Type Các lệnh số học, logic, dịch
- I-Type Các lệnh số học, logic có hằng số
- Các lệnh nhảy và rẽ nhánh
- Chuyển phát biểu If và các biểu thức Boolean
- Các lệnh truy xuất bộ nhớ Load & Store
- Chuyển đổi khối lặp và duyệt mảng
- Các chế độ định địa chỉ

Kiến trúc tập lệnh (ISA)

- Là giao diện chính giữa phần cứng và phần mềm
- Kiến trúc tập lệnh bao gồm:
 - Tập lệnh và định dạng lệnh
 - Kiểu dữ liệu, cách mã hóa và biểu diễn
 - Đối tượng lưu trữ: Thanh ghi (Registers) và Bộ nhớ (Memory)
 - Các chế độ định địa chỉ để truy xuất lệnh và dữ liệu
 - Xử lý các điều kiện ngoại lệ (ví dụ: /0)

Examples	(Versions)	First Introduced in
✧ Intel	(8086, 80386, Pentium, ...)	1978
✧ MIPS	(MIPS I, II, III, IV, V)	1986
✧ PowerPC	(601, 604, ...)	1993

Tập lệnh

- Tập lệnh là ngôn ngữ của bộ vi xử lý
- Kiến trúc tập lệnh MIPS trong môn học:
 - Loại RISC (Reduced Instruction Set Computer)
 - Thiết kế đơn giản và tinh tế
 - Giống với kiến trúc RISC được phát triển giữa thập niên 80 – 90
 - Rất phổ biến, được dùng bởi
 - Silicon Graphics, ATI, Cisco, Sony, ...
 - Phổ biến sau bộ xử lý Intel IA-32: Gần 100 triệu bộ xử lý MIPS được bán trong năm 2002
- Kiến trúc khác: Intel IA-32
 - Loại CISC (Complex Instruction Set Computer)

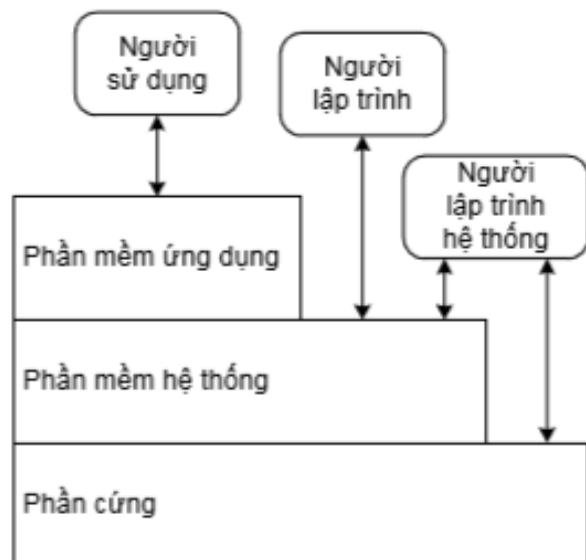
Ví dụ chương trình hợp ngữ MIPS

```
// In what follows R1,R2,R3 are registers, PC is program counter,  
// and addr is some value.  
  
ADD R1,R2,R3      //  $R1 \leftarrow R2 + R3$   
  
ADDI R1,R2,addr   //  $R1 \leftarrow R2 + \text{addr}$   
  
AND R1,R1,R2      //  $R1 \leftarrow R1 \text{ and } R2$  (bit-wise)  
  
JMP addr          //  $PC \leftarrow \text{addr}$   
  
JEQ R1,R2,addr    // IF  $R1 == R2$  THEN  $PC \leftarrow \text{addr}$  ELSE  $PC++$   
  
LOAD R1, addr     //  $R1 \leftarrow \text{RAM}[\text{addr}]$   
  
STORE R1, addr    //  $\text{RAM}[\text{addr}] \leftarrow R1$   
  
NOP              // Do nothing  
  
// Etc. - some 50-300 command variants
```

Khái niệm kiến trúc MT

- Kiến trúc máy tính bao gồm:
 - Kiến trúc tập lệnh (Instruction Set Architecture): nghiên cứu máy tính theo cách nhìn của người lập trình
 - Tổ chức máy tính (Computer Organization) hay Vi kiến trúc (Microarchitecture): nghiên cứu thiết kế máy tính ở mức cao (thiết kế CPU, hệ thống nhớ, cấu trúc bus, ...)
 - Phần cứng (Hardware): nghiên cứu thiết kế logic chi tiết và công nghệ đóng gói của máy tính.
- Cùng một kiến trúc tập lệnh có thể có nhiều sản phẩm (tổ chức, phần cứng) khác nhau

Phân lớp máy tính



- Phần mềm ứng dụng
 - Được viết theo ngôn ngữ bậc cao
- Phần mềm hệ thống
 - Chương trình dịch (Compiler): dịch mã ngôn ngữ bậc cao thành ngôn ngữ máy
 - Hệ điều hành (Operating System)
 - Lập lịch cho các nhiệm vụ và chia sẻ tài nguyên
 - Quản lý bộ nhớ và lưu trữ
 - Điều khiển vào-ra
- Phần cứng
 - Bộ xử lý, bộ nhớ, mô-đun vào-ra

Các mức mã của chương trình

- Ngôn ngữ bậc cao
 - High-level language – HLL
 - Mức trừu tượng gần với vấn đề cần giải quyết
 - Hiệu quả và linh động
- Hợp ngữ
 - Assembly language
 - Mô tả lệnh dưới dạng text
- Ngôn ngữ máy
 - Machine language
 - Mô tả theo phần cứng
 - Các lệnh và dữ liệu được mã hóa theo nhị phân

High-level
language
program
(in C)

```
swap(int v[], int k)
{
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for MIPS)

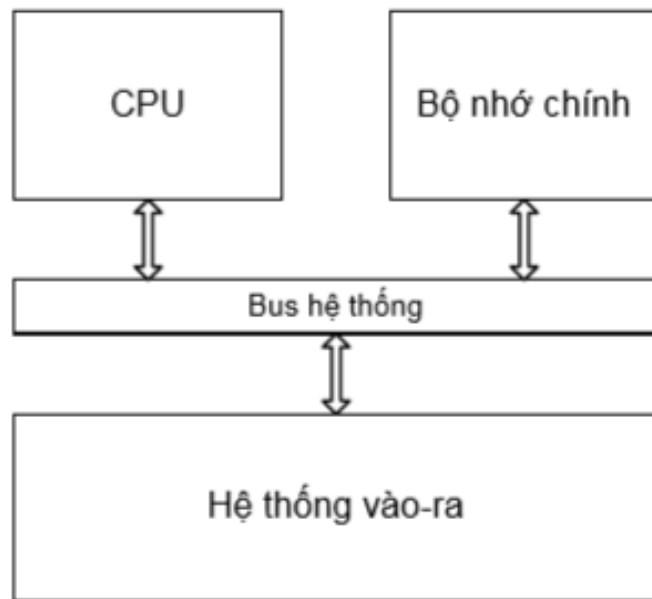
```
swap:
    multi $2, $5, 4
    add $2, $4, $2
    lw $15, 0($2)
    lw $16, 4($2)
    sw $16, 0($2)
    sw $15, 4($2)
    jr $31
```

Assembler

Binary machine
language
program
(for MIPS)

```
000000001010001000000000100011000
00000000100000100001000000100001
10001101111000100000000000000000
100011100001001000000000000000100
101011100001001000000000000000000
101011011110001000000000000000100
0000001111100000000000000000001000
```


Các thành phần cơ bản của MT



- Giống nhau với tất cả các loại máy tính
- **Bộ xử lý trung tâm** (Central Processing Unit – CPU)
 - Điều khiển hoạt động của máy tính và xử lý dữ liệu
- **Bộ nhớ chính** (Main Memory)
 - Chứa các chương trình đang thực hiện
- **Hệ thống vào-ra** (Input/Output)
 - Trao đổi thông tin giữa máy tính với bên ngoài
- **Bus hệ thống** (System bus)
 - Kết nối và vận chuyển thông tin

Next . . .

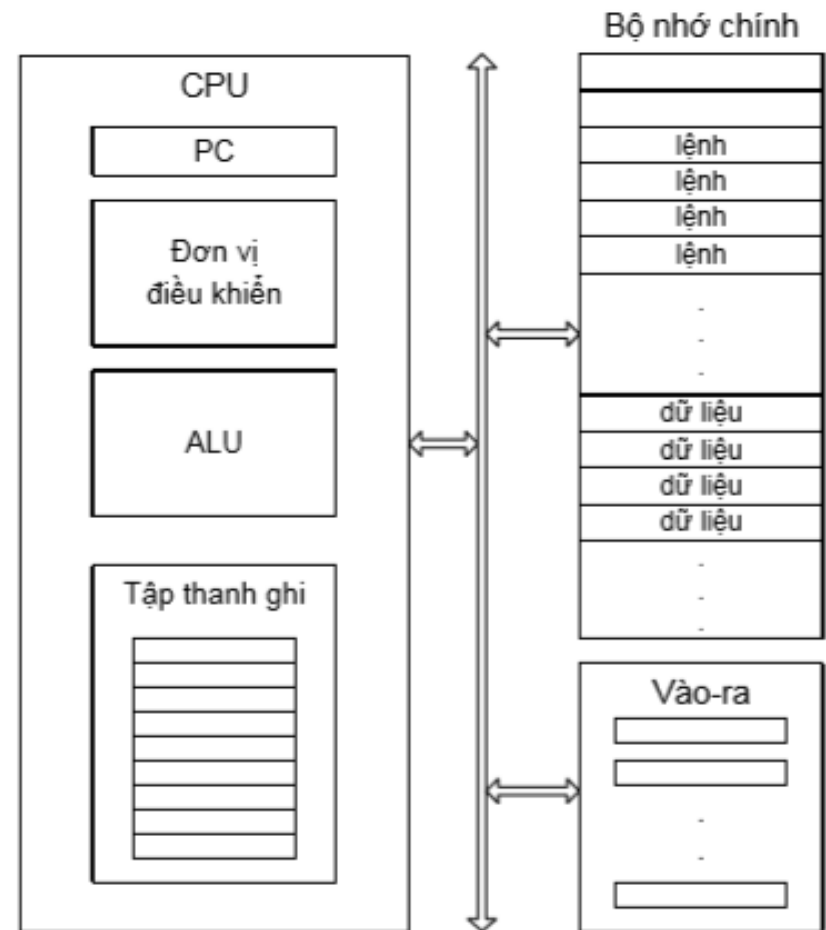
- ❖ Instruction Set Architecture
- ❖ Overview of the MIPS Processor
- ❖ R-Type Arithmetic, Logical, and Shift Instructions
- ❖ I-Type Format and Immediate Constants
- ❖ Jump and Branch Instructions
- ❖ Translating If Statements and Boolean Expressions
- ❖ Load and Store Instructions
- ❖ Translating Loops and Traversing Arrays
- ❖ Alternative Architecture

Kiến trúc tập lệnh

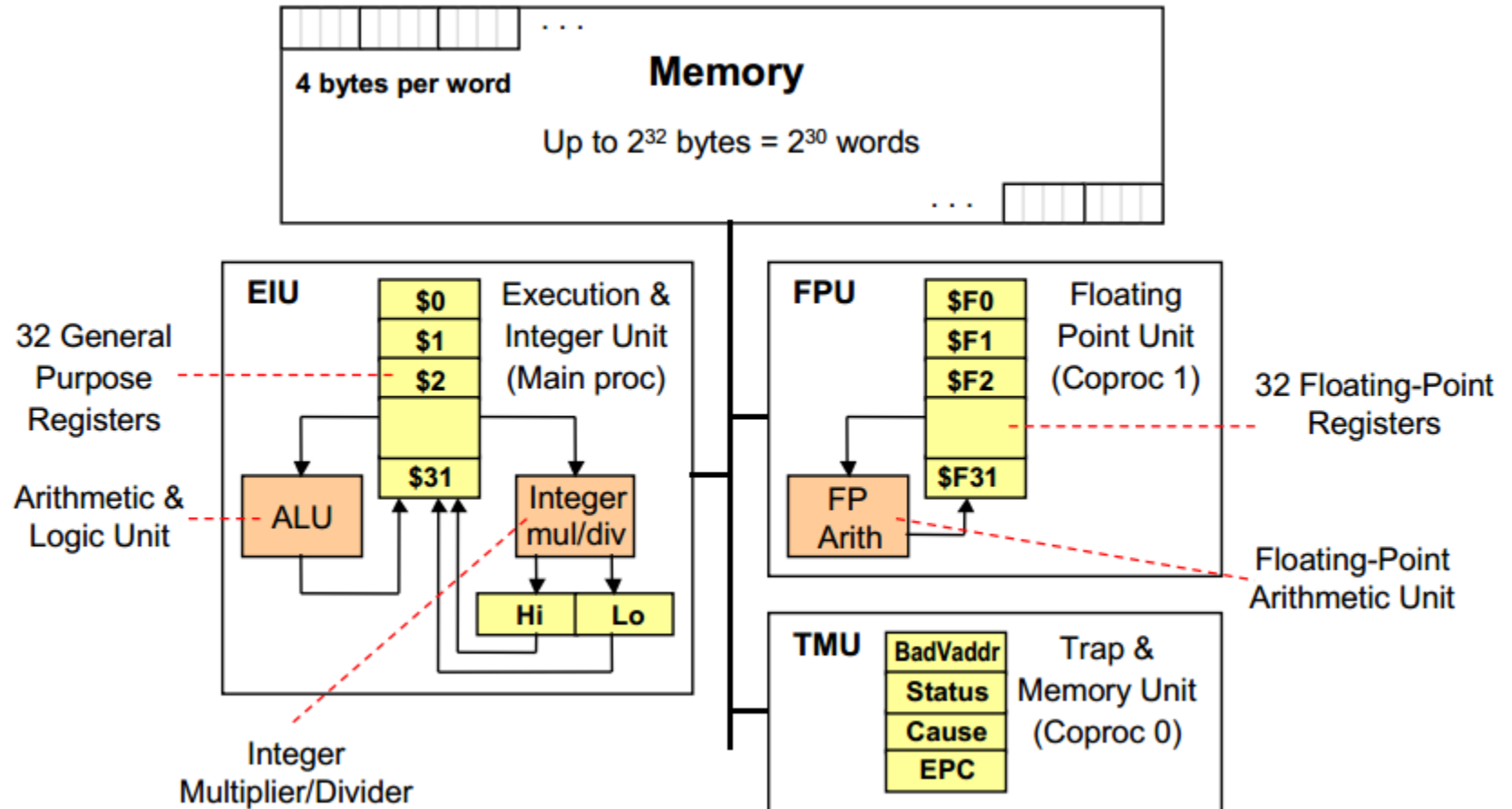
- Kiến trúc tập lệnh (Instruction Set Architecture): cách nhìn máy tính bởi người lập trình
- Vi kiến trúc (Microarchitecture): cách thực hiện kiến trúc tập lệnh bằng phần cứng
- Ngôn ngữ trong máy tính:
 - Hợp ngữ (assembly language):
 - dạng lệnh có thể đọc được bởi con người
 - biểu diễn dạng text
 - Ngôn ngữ máy (machine language):
 - còn gọi là mã máy (machine code)
 - dạng lệnh có thể đọc được bởi máy tính
 - biểu diễn bằng các bit 0 và 1

Mô hình lập trình của MT

- Nhận lệnh từ bộ nhớ
- Giải mã và thực hiện lệnh



Tổng quan kiến trúc VXL MIPS



Các thanh ghi của VXL MIPS

❖ 32 General Purpose Registers (GPRs)

- ✧ 32-bit registers are used in MIPS32
- ✧ Register 0 is always zero
- ✧ Any value written to R0 is discarded

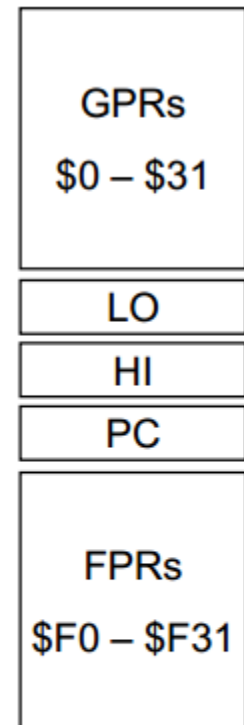
❖ Special-purpose registers LO and HI

- ✧ Hold results of integer multiply and divide

❖ Special-purpose program counter PC

❖ 32 Floating Point Registers (FPRs)

- ✧ Floating Point registers can be either 32-bit or 64-bit
- ✧ A pair of registers is used for double-precision floating-point



Thanh ghi đa năng

❖ 32 General Purpose Registers (GPRs)

- ✧ Assembler uses the dollar notation to name registers
 - \$0 is register 0, \$1 is register 1, ..., and \$31 is register 31
- ✧ All registers are 32-bit wide in MIPS32
- ✧ Register \$0 is always zero
 - Any value written to \$0 is discarded

❖ Software conventions

- ✧ Software defines names to all registers
 - To standardize their use in programs
- ✧ \$8 - \$15 are called \$t0 - \$t7
 - Used for **temporary** values
- ✧ \$16 - \$23 are called \$s0 - \$s7

\$0 = \$zero	\$16 = \$s0
\$1 = \$at	\$17 = \$s1
\$2 = \$v0	\$18 = \$s2
\$3 = \$v1	\$19 = \$s3
\$4 = \$a0	\$20 = \$s4
\$5 = \$a1	\$21 = \$s5
\$6 = \$a2	\$22 = \$s6
\$7 = \$a3	\$23 = \$s7
\$8 = \$t0	\$24 = \$t8
\$9 = \$t1	\$25 = \$t9
\$10 = \$t2	\$26 = \$k0
\$11 = \$t3	\$27 = \$k1
\$12 = \$t4	\$28 = \$gp
\$13 = \$t5	\$29 = \$sp
\$14 = \$t6	\$30 = \$fp
\$15 = \$t7	\$31 = \$ra

Một số đặc điểm của toán hạng thanh ghi

- Đóng vai trò như các biến trong NNLT bậc cao, tuy nhiên khác với biến chỉ có thể giữ giá trị theo kiểu dữ liệu đã được khai báo trước khi sử dụng, thanh ghi không có kiểu, thao tác trên thanh ghi sẽ xác định dữ liệu trong thanh ghi sẽ được đối xử như thế nào.
- Ưu điểm: Bạo xử lý truy xuất thanh ghi nhanh nhất (>1 tỉ lần/giây) vì thanh ghi là 1 phần của phần cứng thường nằm chung mạch với bộ VXL.
- Nhược điểm: Do thanh ghi là 1 phần của phần cứng nên số lượng cố định và hạn chế, do đó khi sử dụng phải khéo léo.
- 8 thanh ghi thường được sử dụng để lưu các biến là \$16->\$23 được đặt tên gợi nhớ như sau \$s0->\$s7 (saved register)

Quy ước tên gọi bộ thanh ghi MIPS

- **Assembler tham khảo thanh ghi bằng tên hoặc số**
 - Lập trình viên thường dùng thanh ghi theo tên
 - Assembler chuyển tên sang số

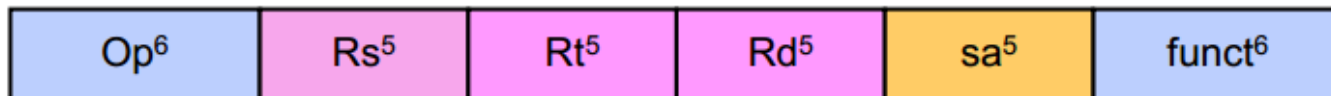
Name	Register	Usage
\$zero	\$0	Always 0 (forced by hardware)
\$at	\$1	Reserved for assembler use
\$v0 – \$v1	\$2 – \$3	Result values of a function
\$a0 – \$a3	\$4 – \$7	Arguments of a function
\$t0 – \$t7	\$8 – \$15	Temporary Values
\$s0 – \$s7	\$16 – \$23	Saved registers (preserved across call)
\$t8 – \$t9	\$24 – \$25	More temporaries
\$k0 – \$k1	\$26 – \$27	Reserved for OS kernel
\$gp	\$28	Global pointer (points to global data)
\$sp	\$29	Stack pointer (points to top of stack)
\$fp	\$30	Frame pointer (points to stack frame)
\$ra	\$31	Return address (used by jal for function call)

3 định dạng lệnh của MIPS ISA

❖ Tất cả các lệnh đều có độ dài 32-bit, có 3 loại:

❖ Register (R-Type)

- ❖ Các lệnh sử dụng 2 toán hạng là giá trị 2 thanh ghi và lưu kết quả vào thanh ghi
- ❖ Op: mã phép toán quy định lệnh



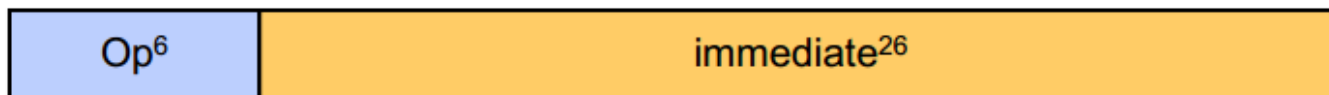
❖ Immediate (I-Type)

- ❖ Hằng số 16-bit là một phần của lệnh



❖ Jump (J-Type)

- ❖ Các lệnh nhảy có định dạng J-Type



Phân loại lệnh trong tập lệnh – nhóm lệnh

❖ Các lệnh số học nguyên (Integer Arithmetic)

- ✧ Các lệnh cộng/trừ, lệnh luận lý (and, or, nor, xor) và lệnh dịch (shift left, shift right)

❖ Các lệnh truy xuất dữ liệu từ bộ nhớ (Data Transfer)

- ✧ Lệnh Load&Store tương ứng thao tác Đọc/Ghi
- ✧ Hỗ trợ dữ liệu byte (1byte), half word (2byte), word (4byte)

❖ Nhảy và rẽ nhánh (Jump and Branch)

- ✧ Các lệnh điều khiển dòng thực thi khác cách tuần tự

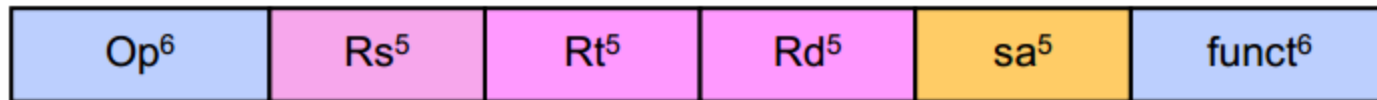
❖ Các lệnh số học số thực (Floating Point Arithmetic)

- ✧ Các lệnh thao tác trên các thanh ghi số thực

❖ Các lệnh phụ

- ✧ Các lệnh hỗ trợ xử lý ngoại lệ (exceptions)
 - ✧ Các lệnh quản lý bộ nhớ
-

R-Type Format



- ❖ **Op**: mã phép toán (opcode)
 - ✧ Cho biết lệnh làm phép toán gì
- ❖ **funct**: function code – mở rộng opcode
 - ✧ Có thể có $2^6 = 64$ functions có thể mở rộng cho một opcode
 - ✧ MIPS sử dụng **opcode 0** để định nghĩa lệnh **loại R-type**
- ❖ Ba thanh ghi toán hạn (Register Operand)
 - ✧ **Rs, Rt**: Hai toán hạn nguồn
 - ✧ **Rd**: Toán hạn đích chứa kết quả
 - ✧ **sa**: Quy định số bit dịch trong các lệnh dịch

Các lệnh cộng/trừ số nguyên

Instruction	Meaning	R-Type Format					
add \$s1, \$s2, \$s3	$\$s1 = \$s2 + \$s3$	op = 0	rs = \$s2	rt = \$s3	rd = \$s1	sa = 0	f = 0x20
addu \$s1, \$s2, \$s3	$\$s1 = \$s2 + \$s3$	op = 0	rs = \$s2	rt = \$s3	rd = \$s1	sa = 0	f = 0x21
sub \$s1, \$s2, \$s3	$\$s1 = \$s2 - \$s3$	op = 0	rs = \$s2	rt = \$s3	rd = \$s1	sa = 0	f = 0x22
subu \$s1, \$s2, \$s3	$\$s1 = \$s2 - \$s3$	op = 0	rs = \$s2	rt = \$s3	rd = \$s1	sa = 0	f = 0x23

❖ add & sub: overflow causes an **arithmetic exception**

✧ In case of overflow, result is not written to destination register

❖ addu & subu: same operation as add & sub

✧ However, no arithmetic exception can occur

✧ **Overflow is ignored**

❖ Many programming languages ignore overflow

✧ The + operator is translated into **addu**

✧ The – operator is translated into **subu**

Ví dụ về lệnh cộng/trừ

❖ Consider the translation of: $f = (g+h) - (i+j)$

❖ Compiler allocates registers to variables

✧ Assume that f, g, h, i , and j are allocated registers $\$s0$ thru $\$s4$

✧ Called the **saved** registers: $\$s0 = \$16, \$s1 = \$17, \dots, \$s7 = \23

❖ Translation of: $f = (g+h) - (i+j)$

`addu $t0, $s1, $s2` # $\$t0 = g + h$

`addu $t1, $s3, $s4` # $\$t1 = i + j$

`subu $s0, $t0, $t1` # $f = (g+h) - (i+j)$

✧ Temporary results are stored in $\$t0 = \8 and $\$t1 = \9

❖ Translate: `addu $t0, $s1, $s2` to binary code

❖ Solution:

op	rs = \$s1	rt = \$s2	rd = \$t0	sa	func
000000	10001	10010	01000	00000	100001

Toán tử Bitwise

❖ Logical bitwise operations: **and**, **or**, **xor**, **nor**

x	y	x and y
0	0	0
0	1	0
1	0	0
1	1	1

x	y	x or y
0	0	0
0	1	1
1	0	1
1	1	1

x	y	x xor y
0	0	0
0	1	1
1	0	1
1	1	0

x	y	x nor y
0	0	1
0	1	0
1	0	0
1	1	0

- ❖ AND instruction is used to clear bits: **x and 0 = 0**
- ❖ OR instruction is used to set bits: **x or 1 = 1**
- ❖ XOR instruction is used to toggle bits: **x xor 1 = not x**
- ❖ NOR instruction can be used as a NOT, how?
 - ✧ **nor \$s1,\$s2,\$s2** is equivalent to **not \$s1,\$s2**

Tập lệnh logic bitwise

Instruction	Meaning	R-Type Format					
and \$s1, \$s2, \$s3	$\$s1 = \$s2 \& \$s3$	op = 0	rs = \$s2	rt = \$s3	rd = \$s1	sa = 0	f = 0x24
or \$s1, \$s2, \$s3	$\$s1 = \$s2 \$s3$	op = 0	rs = \$s2	rt = \$s3	rd = \$s1	sa = 0	f = 0x25
xor \$s1, \$s2, \$s3	$\$s1 = \$s2 \wedge \$s3$	op = 0	rs = \$s2	rt = \$s3	rd = \$s1	sa = 0	f = 0x26
nor \$s1, \$s2, \$s3	$\$s1 = \sim(\$s2 \$s3)$	op = 0	rs = \$s2	rt = \$s3	rd = \$s1	sa = 0	f = 0x27

❖ Examples:

Assume $\$s1 = 0xabcd1234$ and $\$s2 = 0xffff0000$

and \$s0, \$s1, \$s2 # \$s0 = 0xabcd0000

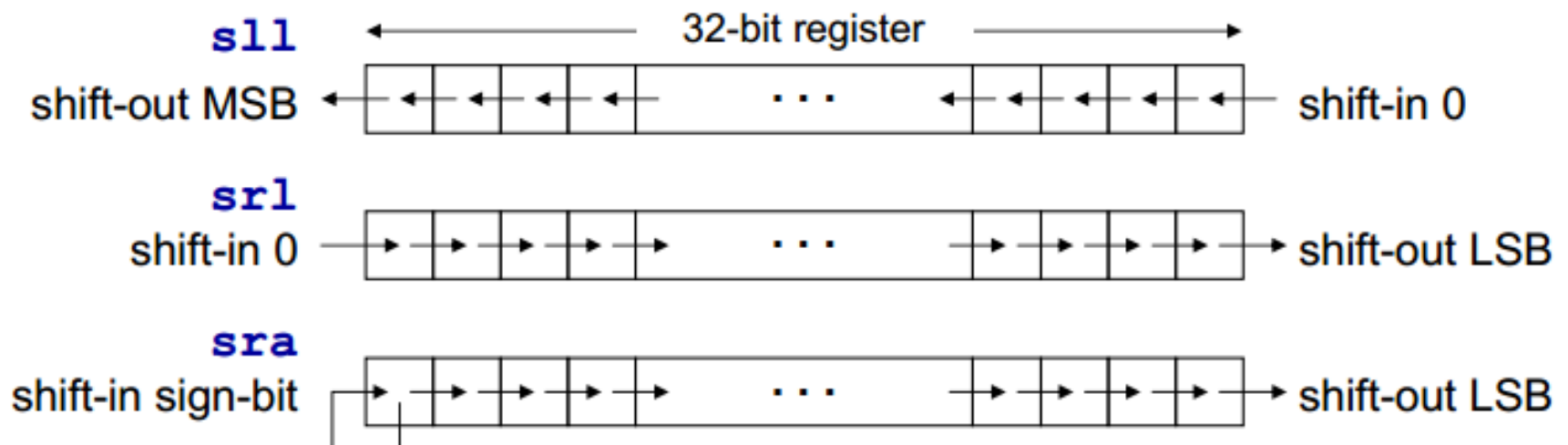
or \$s0, \$s1, \$s2 # \$s0 = 0xffff1234

xor \$s0, \$s1, \$s2 # \$s0 = 0x54321234

nor \$s0, \$s1, \$s2 # \$s0 = 0x0000edcb

Phép dịch

- Phép dịch di chuyển tất cả các bit của một thanh ghi sang trái hoặc sang phải
- Có 3 lệnh dịch số lượng bit cố định: sll, srl, sra
 - sll/srl (shift left/right logical) (số không dấu)
 - Trường “sa” (5-bit shift amount) chỉ số lượng bit được dịch
 - sra (shift right arithmetic) (bảo toàn dấu)
 - Bit dấu (sign-bit) được thêm vào từ bên trái



Các lệnh dịch

Instruction		Meaning	R-Type Format					
sll	\$s1,\$s2,10	\$s1 = \$s2 << 10	op = 0	rs = 0	rt = \$s2	rd = \$s1	sa = 10	f = 0
srl	\$s1,\$s2,10	\$s1 = \$s2 >>> 10	op = 0	rs = 0	rt = \$s2	rd = \$s1	sa = 10	f = 2
sra	\$s1, \$s2, 10	\$s1 = \$s2 >> 10	op = 0	rs = 0	rt = \$s2	rd = \$s1	sa = 10	f = 3
sllv	\$s1,\$s2,\$s3	\$s1 = \$s2 << \$s3	op = 0	rs = \$s3	rt = \$s2	rd = \$s1	sa = 0	f = 4
srlv	\$s1,\$s2,\$s3	\$s1 = \$s2 >>> \$s3	op = 0	rs = \$s3	rt = \$s2	rd = \$s1	sa = 0	f = 6
srav	\$s1,\$s2,\$s3	\$s1 = \$s2 >> \$s3	op = 0	rs = \$s3	rt = \$s2	rd = \$s1	sa = 0	f = 7

- Dịch với số lượng bit thay đổi (variable): sllv, srlv, srav (giống như sll, srl, sra nhưng số lượng bit dịch chứa trong thanh ghi)
- Ví dụ: giả sử \$s2=0xabcd1234, \$s3=16

```

sll  $s1,$s2,8      $s1 = $s2<<8      $s1 = 0xcd123400
sra  $s1,$s2,4      $s1 = $s2>>>4     $s1 = 0xfabcd123
srlv $s1,$s2,$s3    $s1 = $s2>>>>$s3  $s1 = 0x0000abcd

```



op=000000	rs=\$s3=10011	rt=\$s2=10010	rd=\$s1=10001	sa=00000	f=000110
-----------	---------------	---------------	---------------	----------	----------

Phép nhân dựa trên phép dịch (phép chia?)

- Lệnh dịch trái (sll) có thể thực hiện phép nhân
 - Khi số nhân là mũ của 2
- Một số nguyên bất kỳ có thể biểu diễn thành tổng của các số là mũ của 2
 - Ví dụ nhân \$s1 với 36
 - Phân tích 36 thành (4+32) và sử dụng tính chất phân phối của phép nhân
 - $\$s2 = \$s1 * 36 = \$s1 * (4 + 32) = \$s1 * 4 + \$s1 * 32$

```
sll    $t0, $s1, 2      ; $t0 = $s1 * 4
sll    $t1, $s1, 5      ; $t1 = $s1 * 32
addu   $s2, $t0, $t1    ; $s2 = $s1 * 36
```

Ví dụ

- Nhân \$s1 với 26 sử dụng lệnh dịch và cộng
($26=2+8+16$)
- Nhân \$s1 với 31
- Nhân \$1 với 29

```
sll    $t0, $s1, 1      ; $t0 = $s1 * 2
sll    $t1, $s1, 3      ; $t1 = $s1 * 8
addu   $s2, $t0, $t1    ; $s2 = $s1 * 10
sll    $t0, $s1, 4      ; $t0 = $s1 * 16
addu   $s2, $s2, $t0    ; $s2 = $s1 * 26
```

Integer Multiplication & Division

❖ Consider $a \times b$ and a/b where a and b are in $\$s1$ and $\$s2$

✧ Signed multiplication: `mult $s1,$s2`

✧ Unsigned multiplication: `multu $s1,$s2`

✧ Signed division: `div $s1,$s2`

✧ Unsigned division: `divu $s1,$s2`

❖ For multiplication, result is 64 bits

✧ LO = low-order 32-bit and HI = high-order 32-bit

❖ For division

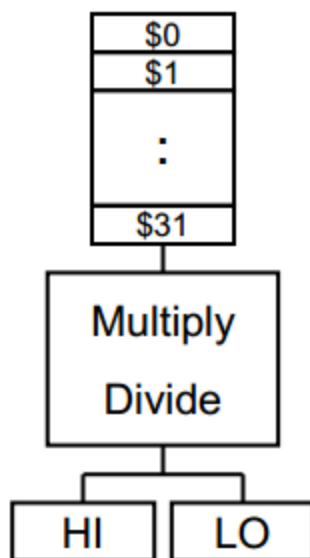
✧ LO = 32-bit quotient and HI = 32-bit remainder

✧ If divisor is 0 then result is unpredictable

❖ Moving data

✧ `mflo rd` (move from LO to rd), `mfhi rd` (move from HI to rd)

✧ `mtlo rs` (move to LO from rs), `mthi rs` (move to HI from rs)



Integer Multiply/Divide Instructions

Instruction	Meaning	Format					
mult rs, rt	hi, lo = rs × rt	op ⁶ = 0	rs ⁵	rt ⁵	0	0	0x18
multu rs, rt	hi, lo = rs × rt	op ⁶ = 0	rs ⁵	rt ⁵	0	0	0x19
div rs, rt	hi, lo = rs / rt	op ⁶ = 0	rs ⁵	rt ⁵	0	0	0x1a
divu rs, rt	hi, lo = rs / rt	op ⁶ = 0	rs ⁵	rt ⁵	0	0	0x1b
mfhi rd	rd = hi	op ⁶ = 0	0	0	rd ⁵	0	0x10
mflo rd	rd = lo	op ⁶ = 0	0	0	rd ⁵	0	0x12
mthi rs	hi = rs	op ⁶ = 0	rs ⁵	0	0	0	0x11
mtlo rs	lo = rs	op ⁶ = 0	rs ⁵	0	0	0	0x13

- ❖ Signed arithmetic: **mult**, **div** (rs and rt are signed)
 - ✧ LO = 32-bit low-order and HI = 32-bit high-order of multiplication
 - ✧ LO = 32-bit quotient and HI = 32-bit remainder of division
- ❖ Unsigned arithmetic: **multu**, **divu** (rs and rt are unsigned)
- ❖ NO arithmetic exception can occur

I-Type

Các lệnh số học, logic có hằng số

Định dạng I-Type

- Hằng số được sử dụng thường xuyên trong chương trình
 - Lệnh dịch thuộc I-type chứa sẵn 5 bit dịch trong lệnh (sa)
- I-Type

Op ⁶	Rs ⁵	Rt ⁵	immediate ¹⁶
-----------------	-----------------	-----------------	-------------------------
- Hằng số 16 bit được lưu trong lệnh
 - Rs thanh ghi toán hạng nguồn
 - Rt thanh ghi toán hạng đích (destination)
- Ví dụ lệnh ALU kiểu I-Type:

✧ Add immediate: `addi $s1, $s2, 5` # `$s1 = $s2 + 5`

✧ OR immediate: `ori $s1, $s2, 5` # `$s1 = $s2 | 5`

Các lệnh số học logic (ALU) kiểu I-Type

Instruction	Meaning	I-Type Format				
addi \$s1, \$s2, 10	$\$s1 = \$s2 + 10$	op = 0x8	rs = \$s2	rt = \$s1	imm ¹⁶ = 10	
addiu \$s1, \$s2, 10	$\$s1 = \$s2 + 10$	op = 0x9	rs = \$s2	rt = \$s1	imm ¹⁶ = 10	
andi \$s1, \$s2, 10	$\$s1 = \$s2 \& 10$	op = 0xc	rs = \$s2	rt = \$s1	imm ¹⁶ = 10	
ori \$s1, \$s2, 10	$\$s1 = \$s2 10$	op = 0xd	rs = \$s2	rt = \$s1	imm ¹⁶ = 10	
xori \$s1, \$s2, 10	$\$s1 = \$s2 \wedge 10$	op = 0xe	rs = \$s2	rt = \$s1	imm ¹⁶ = 10	
lui \$s1, 10	$\$s1 = 10 \ll 16$	op = 0xf	0	rt = \$s1	imm ¹⁶ = 10	

❖ addi: overflow sinh ra **arithmetic exception**

✧ Trong trường hợp overflow, kết quả không được ghi vào thanh ghi đích

❖ addiu: giống lệnh addi nhưng **trần số được bỏ qua**

❖ Hằng số 16 bit trong lệnh addi và addiu là **số có dấu**

✧ Không có lệnh **subi** hoặc **subiu**

❖ Hằng số 16 bit trong lệnh andi, ori, xori là **số không dấu**

Ví dụ về các lệnh ALU kiểu I-Type

❖ Ví dụ: giả sử A, B, C được ánh xạ vào \$s0, \$s1, \$s2

A = B+5; chuyển thành **addiu** \$s0,\$s1,5

C = B-1; chuyển thành **addiu** \$s2,\$s1,-1



op=001001	rs=\$s1=10001	rt=\$s2=10010	imm = -1 = 1111111111111111
-----------	---------------	---------------	-----------------------------

A = B&0xf; chuyển thành **andi** \$s0,\$s1,0xf

C = B|0xf; chuyển thành **ori** \$s2,\$s1,0xf

C = 5; chuyển thành **ori** \$s2,\$zero,5

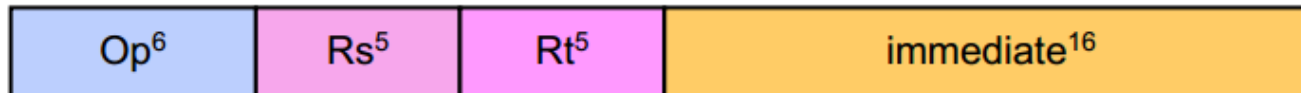
A = B; chuyển thành **ori** \$s0,\$s1,0

❖ Tại sao không có lệnh **subi**? (lệnh **addi** dùng hằng số 16 bit **số có dấu**)

❖ Thanh ghi 0 (**\$zero**) giá trị luôn bằng 0

Hằng số 32 bit?

- ❖ Lệnh I-Type chỉ chứa hằng số 16-bit



- ❖ Làm thế nào để khởi tạo giá trị 32-bit cho một thanh ghi?
- ❖ Không thể có giá trị 32-bit trong lệnh I-Type ☹
- ❖ Gợi ý: dùng nhiều lệnh thay vì một lệnh 😊

✧ Ví dụ: khởi tạo `$s1=0xAC5165D9` (hằng số 32-bit)

```
addiu $s1,$0,0xAC51
sll   $s1,$s1,16
ori   $s1,$s1,0x65D9
```

lui: load upper immediate

```
lui $s1,0xAC51
```

```
ori $s1,$s1,0x65D9
```

	load upper 16 bits	clear lower 16 bits
\$s1=\$17	0xAC51	0x0000
\$s1=\$17	0xAC51	0x65D9

Các lệnh nhảy và rẽ nhánh

Định dạng lệnh J-Type



- ❖ Định dạng J-type áp dụng cho các lệnh nhảy không điều kiện (unconditional jump, giống như lệnh goto):

`j    label    # jump to label`

`. . .`

`label:`

- ❖ Hằng số 26-bit được gắn vào trong lệnh
 - ✧ Hằng số này cho biết địa chỉ nhảy đến
- ❖ Thanh ghi Program Counter (PC) được thay đổi như sau:

✧ Next PC =  least-significant
2 bits are 00

✧ 4 bit cao nhất của PC không đổi

✧ 2 bit cuối luôn bằng 00

Các lệnh rẽ nhánh có điều kiện

❖ MIPS **compare and branch** instructions:

beq *Rs*,*Rt*,*label* branch to **label** if (*Rs* == *Rt*)

bne *Rs*,*Rt*,*label* branch to **label** if (*Rs* != *Rt*)

❖ MIPS **compare to zero & branch** instructions

Compare to zero is used frequently and implemented efficiently

bltz *Rs*,*label* branch to **label** if (*Rs* < 0)

bgtz *Rs*,*label* branch to **label** if (*Rs* > 0)

blez *Rs*,*label* branch to **label** if (*Rs* <= 0)

bgez *Rs*,*label* branch to **label** if (*Rs* >= 0)

❖ No need for **beqz** and **bnez** instructions. Why?

Các lệnh Set on Less Than

- ❖ MIPS cung cấp lệnh **gán bằng 1 khi nhỏ hơn (set on less than)**

slt **rd,rs,rt** if (rs < rt) rd = 1 else rd = 0

sltu **rd,rs,rt** **unsigned <**

slti **rt,rs,im¹⁶** if (rs < im¹⁶) rt = 1 else rt = 0

sltiu **rt,rs,im¹⁶** **unsigned <**

- ❖ So sánh **có dấu / không dấu (Signed / Unsigned)**

Có thể sinh ra các kết quả **khác nhau**

Giả sử **\$s0 = 1** và **\$s1 = -1 = 0xffffffff**

slt **\$t0,\$s0,\$s1** kết quả **\$t0 = 0**

sltu **\$t0,\$s0,\$s1** kết quả **\$t0 = 1**

Các lệnh rẽ nhánh khác

❖ Phần cứng MIPS KHÔNG cung cấp các lệnh rẽ nhánh ...

blt, bltu branch if less than (signed/unsigned)

ble, bleu branch if less or equal (signed/unsigned)

bgt, bgtu branch if greater than (signed/unsigned)

bge, bgeu branch if greater or equal (signed/unsigned)

Có thể thực hiện bằng 2 lệnh

❖ Thực hiện?

❖ Lời giải:

blt \$s0,\$s1,label



slt \$at,\$s0,\$s1

bne \$at,\$zero,label

❖ Thực hiện?

❖ Lời giải:

ble \$s2,\$s3,label



slt \$at,\$s3,\$s2

beq \$at,\$zero,label

Các lệnh giả Pseudo - Instructions

- ❖ Cung cấp bởi assembler và được tự động chuyển đổi sang lệnh có thật

✧ Mục đích để hỗ trợ lập trình hợp ngữ được dễ dàng

Pseudo-Instructions	Các lệnh Thật tương ứng
<code>move \$s1, \$s2</code>	<code>addu \$s1, \$s2, \$zero</code>
<code>not \$s1, \$s2</code>	<code>nor \$s1, \$s2, \$s2</code>
<code>li \$s1, 0xabcd</code>	<code>ori \$s1, \$zero, 0xabcd</code>
<code>li \$s1, 0xabcd1234</code>	<code>lui \$s1, 0xabcd</code> <code>ori \$s1, \$s1, 0x1234</code>
<code>sgt \$s1, \$s2, \$s3</code>	<code>slt \$s1, \$s3, \$s2</code>
<code>blt \$s1, \$s2, label</code>	<code>slt \$at, \$s1, \$s2</code> <code>bne \$at, \$zero, label</code>

- ❖ Assembler dùng thanh ghi `$at` = \$1 trong các chuyển đổi

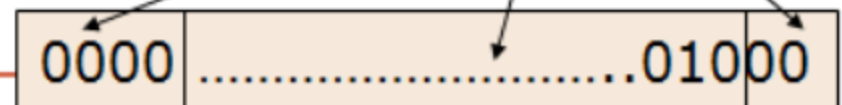
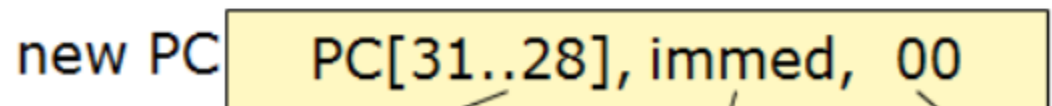
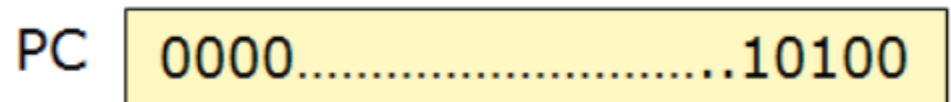
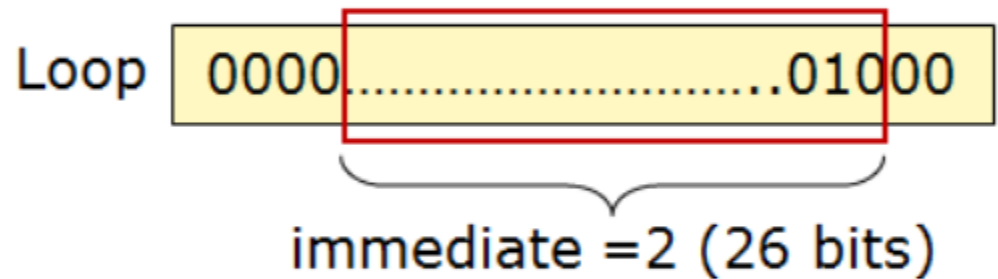
✧ `$at` được gọi là thanh ghi **assembler temporary**

Ví dụ minh họa lệnh beq, j

0000...01000	beq
0000...01001	
0000...01010	
0000...01011	
0000...01100	add
0000...01101	
0000...01110	
0000...01111	
0000...10000	addi
0000...10001	
0000...10010	
0000...10011	
0000...10100	j
0000...10101	
0000...10110	
0000...10111	
0000...11000	
0000...11001	

PC :

Loop: beq \$9, \$0, End #Addr: 8
 add \$8, \$8, \$10 #Addr:12
 addi \$9, \$9, -1 #Addr:16
 j Loop #Addr:20
End: #Addr:24



Các lệnh Jump, Branch, SLT

Instruction		Meaning	Format			
j	label	jump to label	$op^6 = 2$	imm^{26}		
beq	rs, rt, label	branch if (rs == rt)	$op^6 = 4$	rs^5	rt^5	imm^{16}
bne	rs, rt, label	branch if (rs != rt)	$op^6 = 5$	rs^5	rt^5	imm^{16}
blez	rs, label	branch if (rs ≤ 0)	$op^6 = 6$	rs^5	0	imm^{16}
bgtz	rs, label	branch if (rs > 0)	$op^6 = 7$	rs^5	0	imm^{16}
bltz	rs, label	branch if (rs < 0)	$op^6 = 1$	rs^5	0	imm^{16}
bgez	rs, label	branch if (rs ≥ 0)	$op^6 = 1$	rs^5	1	imm^{16}

Instruction		Meaning	Format					
slt	rd, rs, rt	$rd = (rs < rt ? 1 : 0)$	$op^6 = 0$	rs^5	rt^5	rd^5	0	0x2a
sltu	rd, rs, rt	$rd = (rs < rt ? 1 : 0)$	$op^6 = 0$	rs^5	rt^5	rd^5	0	0x2b
slti	rt, rs, imm^{16}	$rt = (rs < imm ? 1 : 0)$	0xa	rs^5	rt^5	imm^{16}		
sltiu	rt, rs, imm^{16}	$rt = (rs < imm ? 1 : 0)$	0xb	rs^5	rt^5	imm^{16}		

Chuyển phát biểu IF và các biểu thức Boolean

Chuyển phát biểu IF

- Xét phát biểu IF sau:

If $(a==b)$ $c = d + e$;

else $c = d - e$;

Giả sử a, b, c, d là các thanh ghi $\$s0, \dots, \$s4$ tương ứng

- Thực hiện chuyển phát biểu IF sang hợp ngữ MIPS:

```
                bne    $s0, $s1, khác
                addu   $s2, $s3, $s4
                j      thoát
khac:           subu   $s2, $s3, $s4
thoat:          ...
```

Biểu thức điều kiện kết hợp AND

- Nếu biểu thức đầu false, biểu thức thứ 2 được bỏ qua

```
if (($s1 > 0) && ($s2 < 0)) {$s3++;}
```

bgtz	\$s1, L1	# biểu thức đầu tiên
j	next	# bỏ qua nếu sai (false)
L1: bltz	\$s2, L2	# biểu thức thứ 2
j	next	# bỏ qua nếu sai
L2: addiu	\$s3, \$s3, 1	# cả 2 đều đúng (true)
next:		

Cách thực hiện tốt hơn

- Số lượng lệnh giảm từ 5 xuống 3

```
if (($s1 > 0) && ($s2 < 0)) {$s3++;}
```

```
# Better Implementation ...  
    blez    $s1, next    # skip if false  
    bgez    $s2, next    # skip if false  
    addiu   $s3,$s3,1    # both are true  
next:
```

Biểu thức IF kết hợp OR

- Nếu biểu thức 1 đúng (true), biểu thức kế tiếp bỏ qua
- bgt, ble, li là các lệnh giả (pseudo-instructions) (được assembler tự động chuyển sang lệnh thật)

```
if (($s1 > $s2) || ($s2 > $s3)) {$s4 = 1;}
```

```
    bgt $s1, $s2, L1      # yes, execute if part
    ble $s2, $s3, next    # no: skip if part
L1:  li  $s4, 1           # set $s4 to 1
next:
```


Ví dụ

- Chuyển biểu thức IF sang hợp ngữ MIPS
- \$s1 và \$s2 là giá trị không dấu

```
if( $s1 <= $s2 ) {  
    $s3 = $s4  
}
```

```
bgtu $s1, $s2, next  
move $s3, $s4  
next:
```

- \$s3, \$s4 và \$s5 là giá trị có dấu

```
if (($s3 <= $s4) &&  
    ($s4 > $s5)) {  
    $s3 = $s4 + $s5  
}
```

```
bgt  $s3, $s4, next  
ble  $s4, $s5, next  
addu $s3, $s4, $s5  
next:
```

Các lệnh truy xuất bộ nhớ

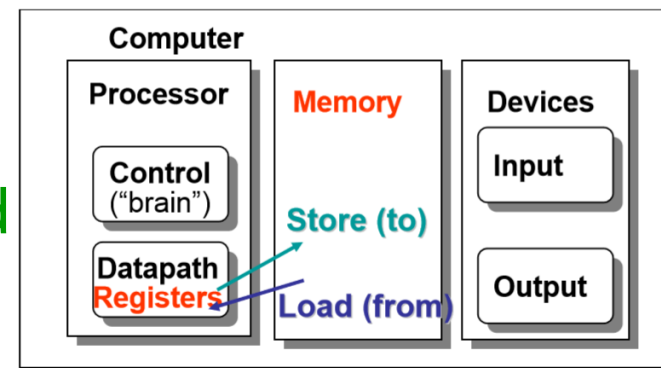
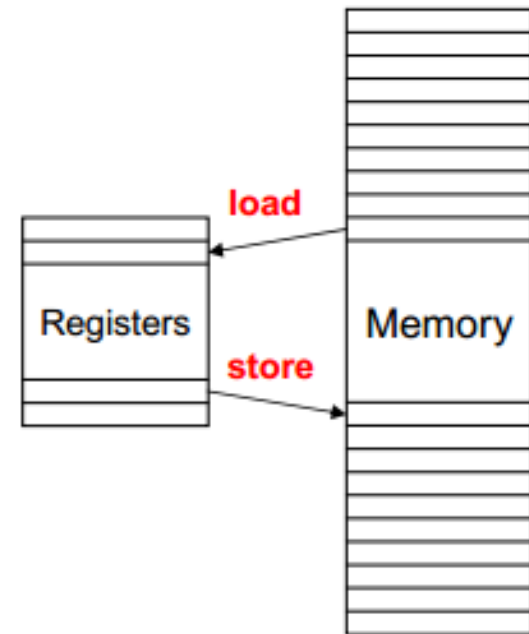
Load & Store

Lệnh Load & Store

- Các lệnh đã học chỉ thao tác trên dữ liệu là số nguyên và dãy bit nằm trong các thanh ghi. Dữ liệu thực tế không đơn giản như vậy. Làm sao để thao tác trên các kiểu dữ liệu phức tạp hơn như mảng hay cấu trúc? Cần bộ nhớ để lưu trữ mọi dữ liệu và lệnh. Bộ xử lý nạp các dữ liệu và lệnh này vào các thanh ghi để xử lý rồi lưu kết quả ngược trở lại bộ nhớ.

Lệnh Load & Store

- Lệnh dùng để di chuyển dữ liệu qua lại giữa **bộ nhớ** và **thanh ghi**
- Chương trình có biến kiểu mảng, đối tượng
- Các biến này được lưu vào bộ nhớ
- Lệnh **Đọc** (Load):
 - Chuyển dữ liệu từ bộ nhớ đến thanh ghi
- Lệnh **Ghi** (Store):
 - Chuyển dữ liệu từ thanh ghi xuống bộ nhớ
- Địa chỉ ô nhớ được chỉ ra bởi lệnh Load và Store

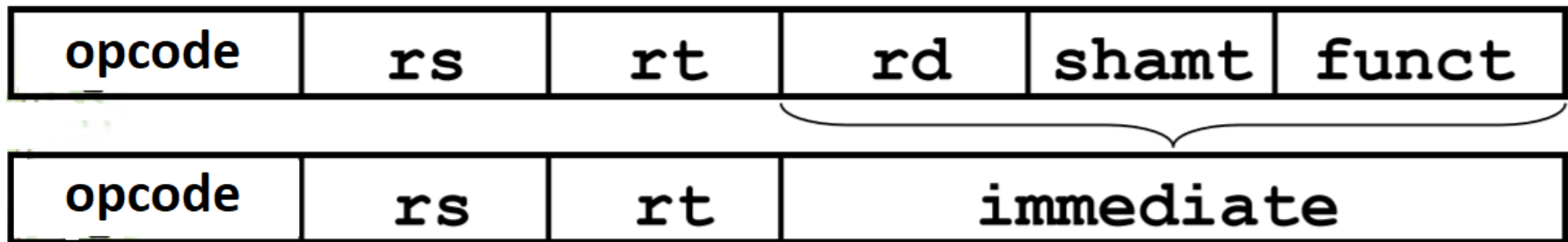


Thao tác trên vùng nhớ

- Bộ nhớ là mảng 1 chiều các ô nhớ có địa chỉ

.	.
.	.
.	.
3	100
2	10
1	101
0	1
Địa chỉ	Dữ liệu

Xác định địa chỉ vùng nhớ



- Để xác định vùng nhớ trong lệnh, cần 2 yếu tố:
 - Một thanh ghi chứa địa chỉ 1 vùng nhớ - địa chỉ cơ sở (base) (xem như con trỏ tới vùng nhớ)
 - Một số nguyên (tính theo byte) (xem như độ dời – offset, từ địa chỉ trong thanh ghi trên).
- Địa chỉ vùng nhớ sẽ được xác định bằng tổng 2 giá trị này
- Ví dụ: 8 (\$t0)
 - Xác định 1 vùng nhớ có địa chỉ bằng giá trị trong thanh ghi \$t0 cộng thêm 8 (byte)

Load và Store dữ liệu Word (4 byte, 32 bit – in MIPS)

- **Lệnh Load Word**

`lw Rt, imm16(Rs)    # Rt ← MEMORY[Rs+imm16]`

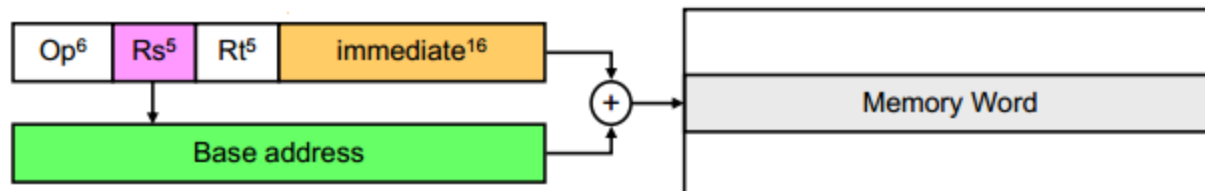
- **Lệnh Store Word**

`sw Rt, imm16(Rs)    # Rt → MEMORY[Rs+imm16]`

- **Cách xác định địa chỉ ô nhớ dùng địa chỉ cơ sở (base) và độ lệch (offset)**

- Memory Address = Rs (**base**) + Immediate¹⁶ (**offset**)
- Độ lệch Immediate¹⁶ được mở rộng dấu thành số 32 bit

Định địa chỉ dùng địa chỉ cơ sở và độ lệch



Ví dụ

- Nạp 1 từ dữ liệu bộ nhớ (load word - lw) vào thanh ghi



lw \$t0, 12 (\$s0)

Lệnh này nạp từ nhớ có địa chỉ ($\$s0 + 12$) vào thanh ghi \$t0

- Lưu 1 từ dữ liệu thanh ghi (store word - sw) vào bộ nhớ



sw \$t0, 12 (\$s0)

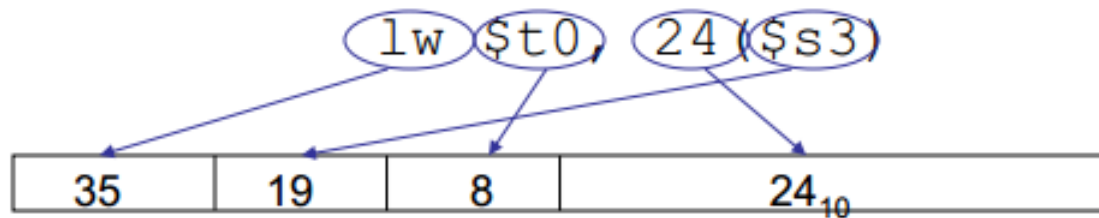
Lệnh này lưu giá trị trong thanh ghi \$t0 vào ô nhớ có địa chỉ ($\$s0 + 12$)

Con trỏ và giá trị

- Một thanh ghi có thể lưu bất kỳ giá trị 32 bit nào, có thể là số nguyên (có dấu/ không dấu), có thể là địa chỉ của 1 vùng nhớ.
 - Nếu ghi (add \$t2, \$t1, \$t0) thì \$t0 và \$t1 lưu giá trị
 - Nếu ghi (lw \$t2, 0 (\$t0)) thì \$t0 chứa 1 địa chỉ (vai trò như 1 con trỏ)

Ví dụ lệnh Load

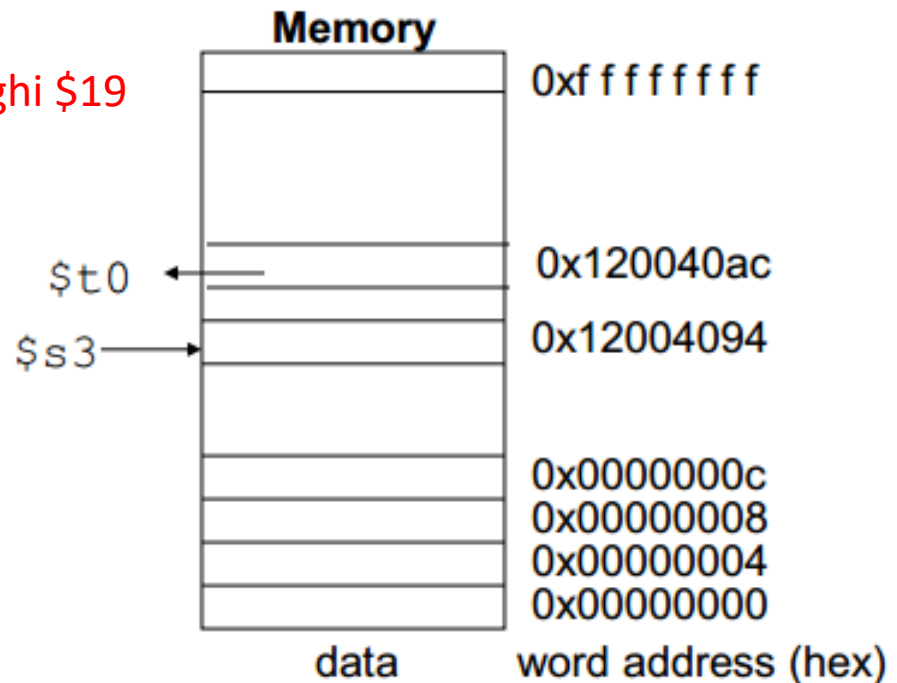
❖ Lệnh Load/Store có định dạng I-Type:



$$24_{10} + \$s3 =$$

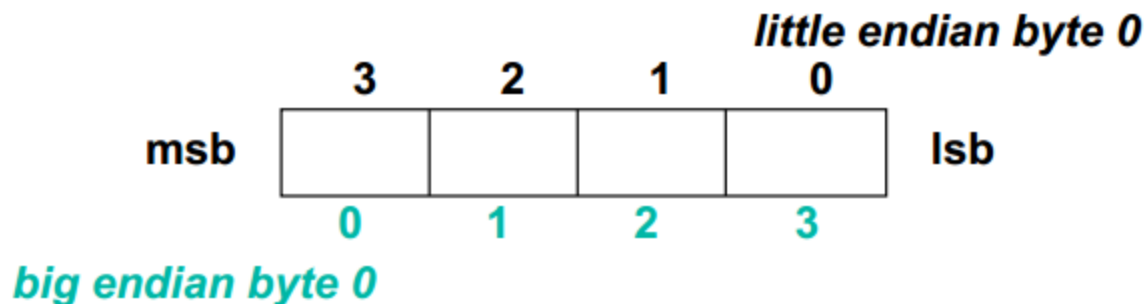
Thanh ghi \$19

$$\begin{array}{r}
 \dots 0001\ 1000 \\
 + \dots 1001\ 0100 \\
 \hline
 \dots 1010\ 1100 = \\
 0x120040ac
 \end{array}$$



Địa chỉ theo Byte

- ❖ Dữ liệu 8-bit bytes vẫn hữu dụng, hầu hết các kiến trúc hỗ trợ định địa chỉ bộ nhớ theo **bytes**
 - ✧ **Alignment restriction** – địa chỉ của một ô nhớ **word** phải là số chẵn cho kích thước một word (4 byte đối với MIPS-32)
- ❖ **Big Endian:** leftmost byte is word address
IBM 360/370, Motorola 68k, **MIPS**, Sparc, HP PA
- ❖ **Little Endian:** rightmost byte is word address
Intel 80x86, DEC Vax, DEC Alpha (Windows NT)



Một số lưu ý về định vị dữ liệu trong bộ nhớ

- MIPS truy xuất dữ liệu trong bộ nhớ theo nguyên tắc Alignment Restriction
- Tuy nhiên, bộ nhớ ngày nay lại được đánh địa chỉ theo từng byte (8 bit)
- Để truy xuất vào một từ nhớ sau 1 từ nhớ thì cần tăng 1 lượng 4 byte chứ không phải 1 byte
- Do đó, đối với lệnh lw, sw thì độ dời (offset) phải là bội số của 4.

Ví dụ

- Giả sử

- A là mảng các từ nhớ
- g: \$s1, h: \$s2, \$s3: địa chỉ bắt đầu của A

- Câu lệnh C:

$g = h + A[5];$

- Được biên dịch thành lệnh máy MIPS như sau:

lw \$t0, 20(\$s3) *#\$t0 gets A[5]*

add \$s1, \$s2, \$t0 *#\$s1 = h+A[5]*

Ví dụ chi tiết dữ liệu lệnh Load

Example

$$\$12 = \text{Memory}[0 + [\$3]]$$

$$\text{Address} = 0 + 0x10010000$$

lw \$12, 0(\$3)

word = 32 bits = 4 B

Data Memory

Register File

\$3	0x10010000
	...
\$12	33 22 11 00

0x10010000

0x10010001

0x10010002

0x10010003

0x00

0x11

0x22

0x33

4 B

Ví dụ chi tiết về lệnh Store

Example

Memory $[0 + r[\$3]] = \12

Address = $0 + 0x10010000$

sw \$12, 0(\$3)

✓ word = 32 bits = 4B

Data Memory

Register File

\$3	0x10010000
	...
\$12	0xAABBCCDD

0x10010000	DD
0x10010001	CC
0x10010002	BB
0x10010003	AA

Ví dụ sử dụng lệnh Load & Store

❖ Chuyển $A[1] = A[2] + 5$ (A là mảng kiểu word)

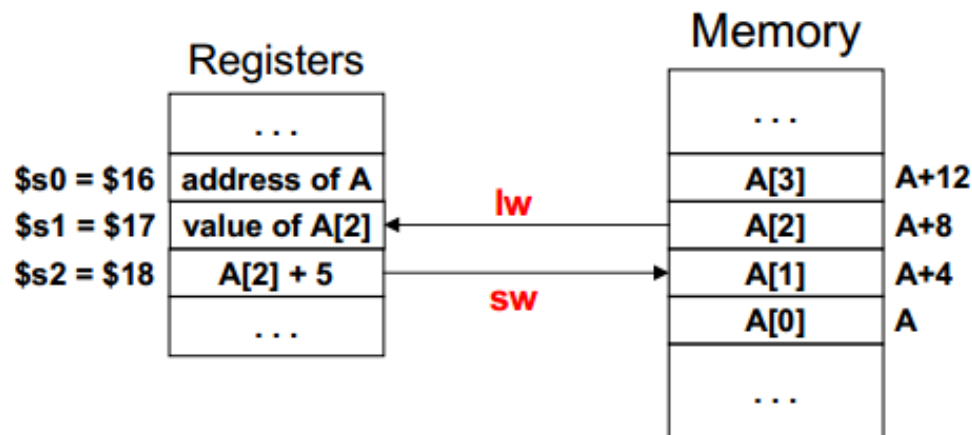
✧ Giả sử địa chỉ mảng A được lưu trong \$s0

`lw        $s1, 8($s0)        # $s1 = A[2]`

`addiu    $s2, $s1, 5        # $s2 = A[2] + 5`

`sw        $s2, 4($s0)        # A[1] = $s2`

❖ Địa chỉ của $a[2]$ và $a[1]$ được nhân 4. Tại sao?



Load/Store Byte và Halfword

❖ MIPS hỗ trợ kiểu dữ liệu:

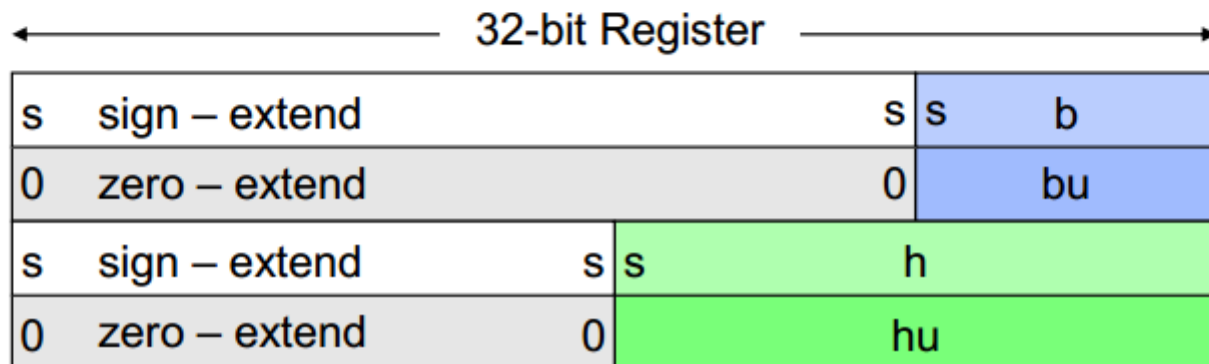
✧ Byte = 8 bits, Halfword = 16 bits, Word = 32 bits

❖ Lệnh Load & store cho bytes và halfwords

✧ lb = load byte, lbu = load byte unsigned, sb = store byte

✧ lh = load half, lhu = load half unsigned, sh = store halfword

❖ Load mở rộng giá trị ô nhớ thành số 32-bit trong thanh ghi



Lệnh nạp, lưu 1 byte nhớ (lb, sb)

- Lb, sb hỗ trợ thao tác với ký tự 1 byte (ASCII)
- Cú pháp: tương tự lw, sw
- Ví dụ: lb \$s0, 3(\$s1)
 - Lệnh này nạp giá trị byte nhớ có địa chỉ (\$s1 + 3) vào byte thấp của thanh ghi \$s0.
 - 24 bit còn lại sẽ có giá trị theo bit dấu của giá trị 1byte (sign-extended)
 - Nếu không muốn các bit còn lại có giá trị theo bit dấu, sử dụng lệnh: lbu (load byte unsigned)



Lệnh nạp, lưu 1/2 từ nhớ (lh, sh)

- Lh, sh hỗ trợ các thao tác với ký tự 2byte (Unicode)
 - Load half (lh): lưu 1/2 từ nhớ (2 byte) vào 2 byte thấp của thanh ghi
 - Store half (sh)
- Cú pháp: tương tự lw, sw

Bảng các lệnh Load & Store

Instruction		Meaning	I-Type Format			
lb	rt, imm ¹⁶ (rs)	rt = MEM[rs+imm ¹⁶]	0x20	rs ⁵	rt ⁵	imm ¹⁶
lh	rt, imm ¹⁶ (rs)	rt = MEM[rs+imm ¹⁶]	0x21	rs ⁵	rt ⁵	imm ¹⁶
lw	rt, imm ¹⁶ (rs)	rt = MEM[rs+imm ¹⁶]	0x23	rs ⁵	rt ⁵	imm ¹⁶
lbu	rt, imm ¹⁶ (rs)	rt = MEM[rs+imm ¹⁶]	0x24	rs ⁵	rt ⁵	imm ¹⁶
lhu	rt, imm ¹⁶ (rs)	rt = MEM[rs+imm ¹⁶]	0x25	rs ⁵	rt ⁵	imm ¹⁶
sb	rt, imm ¹⁶ (rs)	MEM[rs+imm ¹⁶] = rt	0x28	rs ⁵	rt ⁵	imm ¹⁶
sh	rt, imm ¹⁶ (rs)	MEM[rs+imm ¹⁶] = rt	0x29	rs ⁵	rt ⁵	imm ¹⁶
sw	rt, imm ¹⁶ (rs)	MEM[rs+imm ¹⁶] = rt	0x2b	rs ⁵	rt ⁵	imm ¹⁶

Chuyển đổi khối lặp While

Chuyển đổi khối lặp While

❖ Xét phát biểu WHILE:

`i = 0; while (A[i] != k) i = i+1;`

A là mảng số nguyên 4 byte

Giả sử địa chỉ A, i, k tương ứng \$s0, \$s1, \$s2

Memory

...	
A[i]	A+4×i
...	
A[2]	A+8
A[1]	A+4
A[0]	A
...	

❖ Chuyển phát biểu WHILE?

```
        xor    $s1, $s1, $s1    # i = 0
        move   $t0, $s0         # $t0 = address A
loop:    lw     $t1, 0($t0)      # $t1 = A[i]
        beq    $t1, $s2, exit   # exit if (A[i]== k)
        addiu  $s1, $s1, 1      # i = i+1
        sll    $t0, $s1, 2      # $t0 = 4*i
        addu   $t0, $s0, $t0    # $t0 = address A[i]
        j      loop
exit:    . . .
```

Sử dụng con trỏ để duyệt Arrays

❖ Xét phát biểu WHILE:

```
i = 0; while (A[i] != k) i = i+1;
```

A là mảng số nguyên 4 byte

Giả sử địa chỉ A, i, k tương ứng \$s0, \$s1, \$s2

❖ Sử dụng **con trỏ (Pointer)** để duyệt mảng A

Pointer được tăng lên 4 (faster than indexing)

```
        move    $t0, $s0           # $t0 = $s0 = addr A
        j       cond              # test condition
loop:    addiu   $s1, $s1, 1        # i = i+1
        addiu   $t0, $t0, 4        # point to next
cond:    lw      $t1, 0($t0)        # $t1 = A[i]
        bne     $t1, $s2, loop     # loop if A[i] != k
```

❖ Chỉ còn 4 lệnh (thay vì 6) trong thân vòng lặp

Copying a String

- Copy chuỗi nguồn sang chuỗi đích
- Địa chỉ của chuỗi nguồn là \$s0 và đích là \$s1
- Chuỗi kết thúc bằng ký tự null

```
i = 0;  
do {target[i]=source[i]; i++;} while (source[i]!=0);
```

```
        move    $t0, $s0          # $t0 = pointer to source  
        move    $t1, $s1          # $t1 = pointer to target  
L1:  lb        $t2, 0($t0)        # load byte into $t2  
        sb        $t2, 0($t1)      # store byte into target  
        addiu    $t0, $t0, 1       # increment source pointer  
        addiu    $t1, $t1, 1       # increment target pointer  
        bne      $t2, $zero, L1    # loop until NULL char
```


Tính tổng của một mảng nguyên

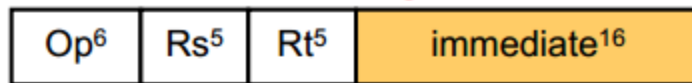
```
sum = 0;  
for (i=0; i<n; i++) sum = sum + A[i];
```

Giả sử \$s0 = địa chỉ của A, \$s1 = kích thước mảng = n

```
move    $t0, $s0           # $t0 = address A[i]  
xor     $t1, $t1, $t1      # $t1 = i = 0  
xor     $s2, $s2, $s2      # $s2 = sum = 0  
L1: lw   $t2, 0($t0)        # $t2 = A[i]  
addu    $s2, $s2, $t2      # sum = sum + A[i]  
addiu   $t0, $t0, 4         # point to next A[i]  
addiu   $t1, $t1, 1         # i++  
bne     $t1, $s1, L1        # loop if (i != n)
```

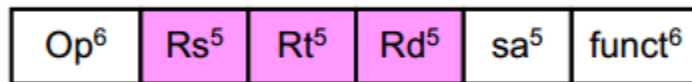
Các chế độ định địa chỉ

Immediate Addressing

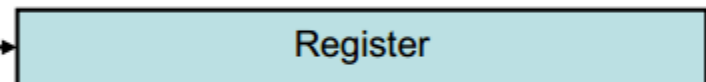


Operand is a constant

Register Addressing



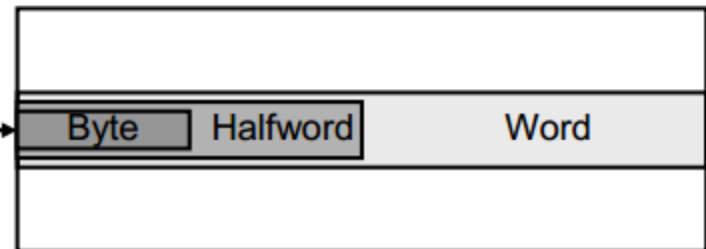
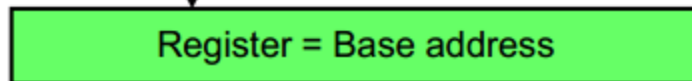
Operand is in a register



Base or Displacement Addressing

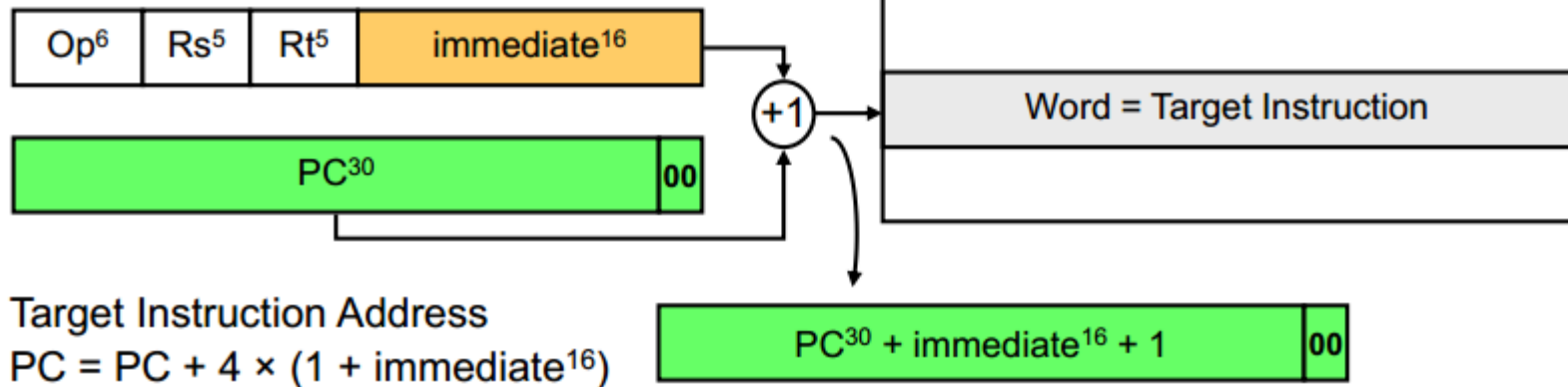


Operand is in memory (load/store)

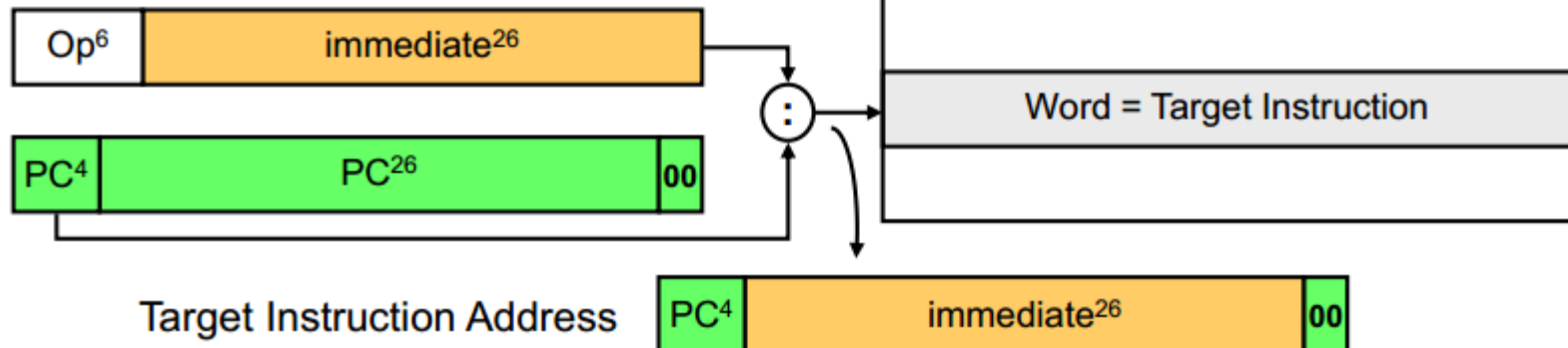


Chế độ định chỉ chỉ của lệnh J và B

PC-Relative Addressing



Pseudo-direct Addressing



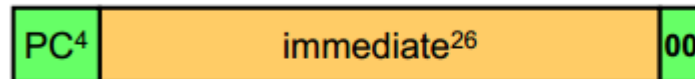
Giới hạn của lệnh J và B

❖ Phạm vi địa chỉ của lệnh Jump = 2^{26} lệnh = 256 MB

✧ Nhảy đến vị trí không quá 2^{26} lệnh hoặc 256 MB

✧ 4 bit cao của PC không đổi

Địa chỉ đích của lệnh Jump



❖ Phạm vi địa chỉ của lệnh rẽ nhánh

✧ Lệnh rẽ nhánh có định dạng I-Type (16-bit hằng số)

✧ Định địa chỉ tương đối với PC:



▪ Địa chỉ đích = PC + 4 × (1 + immediate¹⁶)

▪ Là số lệnh tính từ vị trí lệnh kế của lệnh rẽ nhánh

▪ Hằng số dương => Forward, Hằng số âm => Backward

▪ Khoảng cách xa nhất $\pm 2^{15}$ lệnh (thông thường lệnh rẽ nhánh có đích chỉ đích lân cận vị trí lệnh)